

OAuth 2.0 Authentication Vulnerabilities

OAuth closes the loop on several guides at once — it's the exact mechanism your Open Redirect guide's Section 6c chaining example referenced (a weak `redirect_uri` plus an open redirect stealing an OAuth token), it commonly issues JWTs as its tokens (your JWT guide), and it's fundamentally a *delegated authorization* framework that gets bent into an *authentication* mechanism — a distinction that turns out to be the root of nearly every vulnerability in this guide.

1. What OAuth Actually Is

OAuth is an authorization framework that lets one website (a **client application**) request limited access to a user's data held by another website (the **OAuth service provider**), **without the user ever handing over their password** to the client application. The classic example: an app that wants to suggest contacts by reading your email contacts list, without you ever typing your email password into that app.

The three parties in every OAuth flow:

```
Client application    → the site requesting access to your data
Resource owner        → you, the user whose data is being accessed
OAuth service provider → the site that actually holds your data
                        (provides both an authorization server
                        and a resource server)
```

2. The Basic OAuth Flow

1. Client app requests access to a subset of your data, specifying which "grant type" (flow) and what level of access ("scope")
2. You log in to the OAuth service and explicitly consent to that access
3. Client app receives an access token proving it has your permission
4. Client app uses that token to call the resource server's API and actually fetch the data

3. OAuth for *Authentication* — Where It Gets Risky

OAuth wasn't originally designed to answer "who is this user," only "what data can this app access" — but it's become the standard mechanism behind "Log in with Google/Facebook/GitHub" buttons everywhere, essentially functioning like SAML-based single sign-on from the end-user's perspective.

1. User clicks "Log in with [social media]"
2. Client app requests access to identifying data (often just email)
3. Client app fetches that data from the OAuth provider's /userinfo endpoint
4. Client app uses that identifying data IN PLACE OF a username, and the access token IN PLACE OF a password, to log the user in

This substitution is the root of the entire vulnerability category: OAuth's security model was built around protecting *data access*, not around the guarantees a genuine authentication system needs — and bolting authentication on top of it, without extra rigor, is exactly where things go wrong.

4. Grant Types Worth Knowing

Authorization code grant — the more secure, more common flow: the browser only ever sees a short-lived **code**, which the client app's *server* then exchanges for the actual access token via a secure, direct server-to-server request (never exposed to the browser).

Implicit grant — a simplified flow originally intended for browser-only apps (no server-side backend): the **access token itself** is sent directly through the browser as a URL fragment, with no server-side exchange step at all. This is structurally riskier since the token is exposed to browser-level interception the whole way through, and is exactly the flow behind Section 6's vulnerability.

5. Identifying OAuth Authentication in the Wild

Look for: "Log in with [social media/provider]" buttons

Proxy the flow through Burp/ZAP (per your OWASP ZAP guide) and watch for a request to an /authorization endpoint containing:

```
GET /authorization?client_id=12345&redirect_uri=https://client-app.com/callback
&response_type=token&scope=openid%20profile&state=ae13d489bd00e3c24
```

```
client_id      → identifies which client application is requesting access
redirect_uri   → where the OAuth service should send the result
response_type  → "code" (authorization code flow) or "token" (implicit flow)
scope         → what data/access is being requested
state         → CSRF protection (Section 7 – often missing)
```

Recon step worth always trying — most OAuth providers expose a standard configuration endpoint that reveals implementation details, supported features, and endpoint names you'd otherwise have to guess:

```
GET /.well-known/oauth-authorization-server
GET /.well-known/openid-configuration
```

This is genuinely the same "check the standard well-known path first" instinct as your Google Dorking guide's exposed-config-file dorks — just applied to a documented, legitimate OAuth discovery endpoint instead of an accidental exposure.

6. Vulnerability 1 — Improper Implementation of the Implicit Grant

Since the implicit flow sends the access token through the browser via a URL fragment, the client app's JavaScript must read that token and typically submits it to its own backend server (via a `POST` request) to establish a persistent session cookie — the server has no independent secret to check this data against, so **it implicitly trusts whatever the browser sends it**.

```
POST /authenticate
{ "user_id": "carlos", "token": "z0y9x8w7v6u5" }
```

The vulnerability: if the server doesn't properly verify that the submitted token *actually* matches/authorizes the submitted `user_id`, an attacker can simply **change the `user_id` field** in this request — directly impersonating any user, with zero need to steal an actual valid token at all.

7. Vulnerability 2 — Flawed CSRF Protection (Missing `state` Parameter)

The `state` parameter is meant to carry an unguessable value tied to the user's session when the OAuth flow begins, round-tripped through the entire flow specifically to function as a **CSRF token** for the client application.

If `state` is missing: an attacker can initiate an OAuth flow **on their own account**, then trick a victim's browser into completing it — a classic CSRF pattern, applied to authentication itself:

Scenario: a site lets users log in either via password OR by linking a social media account via OAuth

Attack: attacker starts the OAuth linking flow using THEIR OWN social media account, captures the resulting request, and tricks the victim into completing it (e.g., via an auto-submitting form on a malicious page)

Result: the victim's account on the client application gets bound to the ATTACKER'S social media account — the attacker can now log in AS THE VICTIM any time, simply by authenticating with their own social media credentials

Even on OAuth-only login sites (no separate password mechanism), a missing `state` parameter still enables a **login CSRF** — tricking a victim into unknowingly logging into the *attacker's* account, which can be used to plant malicious data the victim might unknowingly trust or interact with.

8. Vulnerability 3 — Leaking Authorization Codes and Access Tokens (The Headline Attack)

This is widely considered OAuth's most infamous vulnerability class, and directly reuses your Open Redirect guide's chaining methodology.

Root cause: the `redirect_uri` parameter tells the OAuth service where to send the resulting code/token. If the OAuth service doesn't properly validate this URI, an attacker can construct a CSRF-like attack tricking a **victim's** browser into initiating an OAuth flow that sends the resulting code/token to an **attacker-controlled** `redirect_uri` instead.

Attacker crafts:

```
https://oauth-provider.com/authorization?client_id=12345&
  redirect_uri=https://attacker.com/callback&response_type=code&
  scope=openid%20profile
```

Victim (already logged into the OAuth provider) clicks this link,

which they were tricked into via phishing/CSRF — their code/token gets sent directly to attacker.com instead of the legitimate app

For the authorization code flow specifically, this is even more severe than it first appears: the attacker doesn't need the client secret or the eventual access token at all — they just need to relay the **stolen code** to the legitimate client app's genuine `/callback` endpoint themselves. The client app completes the code-for-token exchange normally, believing it's talking to the real victim, and logs the **attacker** in as the victim.

Important nuance: `state` / `nonce` protection does **not** prevent this specific attack, since the attacker generates their own fresh, valid `state` value from their own browser when initiating the malicious flow — this is a distinct vulnerability from Section 7's missing-`state` CSRF, and defeating it requires the OAuth service to independently validate `redirect_uri` server-side, not rely on `state` alone.

The proper server-side defense: requiring the client app to resend `redirect_uri` again during the code-for-token exchange step itself (a secure server-to-server request), and rejecting the exchange if it doesn't match the original value — since this second exchange happens entirely server-side, the attacker never gets a chance to control it.

9. Bypassing `redirect_uri` Validation — Practical Techniques

Even when an OAuth service does validate `redirect_uri` against a registered allowlist, the validation logic itself is frequently exploitable:

a) Prefix-matching weaknesses

If validation just checks the URI STARTS WITH an approved domain, try adding arbitrary subdirectories, query parameters, or fragments to see what passes without triggering an error.

b) Parser discrepancy tricks — directly reuses your SSRF/CORS knowledge

```
https://default-host.com &@foo.evil-user.net#@bar.evil-user.net/
```

Different components of the OAuth service (the validation logic vs. the actual redirect logic) may parse this same string differently — exactly the class of technique covered in your XXE guide's SSRF discussion and referenced again in the CORS-parsing context; worth revisiting those exact bypass patterns here.

c) Parameter pollution

```
?redirect_uri=client-app.com/callback&redirect_uri=evil-user.net
```

If the validating component reads the *first* `redirect_uri` value but the redirecting component actually uses the *last* one (or vice versa), you've found a bypass through pure parser inconsistency.

d) `localhost` special-casing

Some servers grant special leniency to `localhost`-prefixed URIs (since they're common in development) — occasionally this leniency accidentally survives into production, letting a domain like `localhost.evil-user.net` slip past validation that assumes anything starting with `localhost` is safe.

e) Interaction between multiple parameters

Changing `response_mode` from `query` to `fragment` can sometimes completely alter how `redirect_uri` gets parsed downstream — a good reminder (echoing your SQLi/XXE guides) that testing parameters in isolation misses vulnerabilities that only emerge from specific *combinations*.

10. Stealing Codes/Tokens via a Proxy Page (When Direct External Redirect Fails)

Against well-defended targets where an external `redirect_uri` is flatly rejected, the next move is finding **any other page on the already-whitelisted domain** that can leak the code/token onward:

```
Directory traversal trick against the registered callback path:  
https://client-app.com/oauth/callback/../../../../example/path  
→ may resolve server-side to: https://client-app.com/example/path
```

Once you've found which other whitelisted-domain pages you can set as `redirect_uri`, audit **those** pages for a secondary vulnerability that lets you actually exfiltrate the code (found in query parameters) or token (found in the URL fragment):

Open redirect on that page	→ forwards the victim (and their code/token) onward to an attacker-controlled domain – this is EXACTLY the chain from your Open Redirect guide's Section 6c
Insecure JavaScript/web-messaging handling	→ a script that reads query params or URL fragments and does something exploitable with them (potentially a multi-step "gadget chain" through several scripts)
XSS on that page	→ even though session cookies are often HttpOnly-protected (your Cookies guide, Section 4) and thus unreachable via XSS, an OAuth code/token sitting in the URL is NOT cookie-protected at all – XSS can read and exfiltrate it directly, which meaningfully INCREASES the real severity of what might otherwise be dismissed as a low-impact XSS
HTML injection (no JS execution needed)	→ even where CSP blocks script execution, injecting a bare <code></code> tag can leak the code via the Referer header on some browsers (Firefox notably includes the full query string) – the exact same Referer-leak mechanism called out in your Authentication guide's password-reset-token section and your Sessions guide

Why the implicit grant makes this even worse: since the entire flow happens client-side, a stolen implicit-flow access token doesn't just let an attacker log in as the victim on the client app — they can make their **own direct API calls** to the OAuth provider's resource server using that token, potentially accessing sensitive data the client app's own UI never even exposes.

11. Vulnerability 4 — Flawed Scope Validation ("Scope Upgrade")

Users approve access based on a specific requested **scope** — the resulting token should only grant that approved level of access. Flawed server-side validation can let an attacker "upgrade" a token to broader permissions than were ever actually approved.

Authorization code flow scope upgrade: since the attacker controls their own registered client application, they can simply add extra scopes to the **code-for-token exchange request** itself, after the user only approved a narrower scope initially:

```
POST /token
client_id=12345&client_secret=SECRET&code=a1b2c3d4&
grant_type=authorization_code&scope=openid%20email%20profile
                                     ^^^^^^^ never approved by the user
```

If the server doesn't check this against the *original* authorization request's scope, it may simply issue a token with the broader, never-approved access.

Implicit flow scope upgrade: since the token itself passes through the browser, an attacker who's stolen someone else's token (via Section 10's techniques) can manually add an extra `scope` parameter when calling the `/userinfo` endpoint directly — if the server doesn't validate the requested scope against what the token was actually issued for, additional data can be accessed with no further user approval needed at all.

12. Vulnerability 5 — Unverified User Registration

OAuth-based authentication implicitly trusts that the identifying data (typically an email address) held by the OAuth provider is accurate. If the OAuth provider itself allows registering an account **without verifying** that email address, an attacker can register with the provider using a **victim's known email address**, then use that fraudulent-but-technically-valid OAuth account to log into any client application that trusts this provider's identity data — impersonating the victim on every downstream site relying on that OAuth provider, without ever touching the victim's actual accounts anywhere.

13. OpenID Connect — The Authentication-Specific Extension

OpenID Connect (OIDC) is a layer built on top of OAuth specifically to add proper authentication features that plain OAuth was never designed for — most notably the **ID token**, which (unlike a plain OAuth access token) is a **signed JWT** containing verified claims about the user's identity.

This directly connects to your JWT guide: an OIDC ID token is exactly the kind of JWT your JWT guide covered in depth — meaning every attack from that guide (signature bypass, `alg: none`, `jwk / jku / kid` header injection, algorithm confusion) is directly applicable to auditing an OIDC-based authentication flow. Worth treating "this app uses OpenID Connect" as a direct trigger to revisit your JWT guide's full checklist against the ID token specifically.

Two additional OIDC-specific issues worth knowing:

```
Unprotected dynamic client registration → some OIDC providers let ANY
                                         site register itself as a
                                         valid client with no vetting,
                                         which an attacker can abuse
                                         to register a malicious
                                         client application that
                                         then participates in flows
                                         against real users

Authorization requests "by reference" → instead of sending all
                                         authorization parameters
                                         directly, some OIDC flows
                                         let a request reference
                                         a URL where those parameters
                                         are hosted instead - if
                                         that referenced URL isn't
                                         itself properly validated,
                                         it opens up another SSRF-
                                         adjacent trust issue similar
                                         in spirit to the jku header
                                         injection from your JWT guide
```

14. Practical Testing Workflow

1. Identify OAuth usage via the `/authorization` request pattern
2. Check well-known config endpoints for hidden feature/attack surface
3. Test for a missing state parameter → CSRF-style account linking/login CSRF risk
4. Systematically test `redirect_uri` validation:
 - Try external domains directly
 - Try prefix-matching bypasses (extra paths/params/fragments)
 - Try parser-discrepancy tricks (`&@`, `#@`)

- Try parameter pollution (duplicate redirect_uri)
 - Try localhost-prefixed tricks
 - Try changing response_mode alongside redirect_uri
5. If direct external redirect is blocked, find OTHER whitelisted-domain pages reachable via directory traversal or path variation, then audit THOSE for open redirect / XSS / HTML injection that could leak the code/token onward
 6. Test scope validation - can you request additional scopes during the token exchange, or when calling /userinfo directly with a stolen/obtained token?
 7. If OpenID Connect is in use, pull the ID token and run your full JWT guide checklist against it

15. Prevention

For OAuth service providers:

- Strictly validate redirect_uri against an EXACT allowlist - no prefix-matching, no partial-string logic
- Require redirect_uri again during the code-for-token exchange, and reject mismatches - this closes Section 8's core attack, since the exchange happens server-to-server, out of attacker reach
- Properly validate scope during token exchange/refresh against the ORIGINAL authorization request
- Verify user-provided details (especially email) before trusting them for identity purposes
- If supporting dynamic client registration, require vetting/approval

For OAuth client applications:

- Always use and properly validate the state parameter as genuine CSRF protection
- For implicit-grant implementations, ensure the server-side session creation step properly verifies the token actually corresponds to the claimed user - never implicitly trust submitted user_id/token pairings
- Prefer the authorization code flow over implicit wherever feasible, given its stronger server-to-server exchange model
- If using OpenID Connect, apply your JWT guide's full signature-verification checklist to the ID token

Kill Chain / ATT&CK / CWE / OWASP — Full Mapping

Framework	Mapping
CWE	CWE-352 (CSRF, for missing <code>state</code>), CWE-601 (Open Redirect, when chained per Section 10), CWE-346 (Origin Validation Error, for <code>redirect_uri</code> bypasses)
OWASP Top 10:2025	A07:2025 Authentication Failures (OAuth-as-authentication is explicitly this category); A01:2025 Broken Access Control when scope-upgrade or SSRF-adjacent redirect issues are involved
CVSS	Token/code theft routinely scores High/Critical — full account takeover with PR:N/UI:R (victim interaction needed to trigger the CSRF-style redirect) is a common profile
Cyber Kill Chain	Delivery (the phishing/CSRF link that initiates the malicious flow) → Exploitation (the actual token/code theft)
MITRE ATT&CK	T1528 (Steal Application Access Token) — the technique specifically named for exactly this attack category

Quick Reference Cheat Sheet

Identify: GET /authorization?client_id=...&redirect_uri=...&response_type=...&state=...

Recon: /.well-known/oauth-authorization-server
 /.well-known/openid-configuration

Missing state → CSRF-style account-linking / login CSRF

redirect_uri bypass:

extra paths/params	https://trusted.com/extra/path
parser discrepancy	https://trusted.com &@evil.com#@evil.com/
parameter pollution	redirect_uri=trusted.com&redirect_uri=evil.com
localhost trick	localhost.evil.com

Proxy-page chaining:

directory traversal on callback path → find other whitelisted pages
→ open redirect / XSS / HTML injection (Referer leak) on THOSE pages

Scope upgrade:

auth code flow → add scope= during YOUR OWN token exchange request
implicit flow → add scope= when calling /userinfo with a stolen token

OpenID Connect ID token = a JWT → apply full JWT guide checklist